Advanced Python — Python for Data Science

Complete course + cheat sheet for the DS
Chapter 0 (Python basics) · Chapter 1 (NumPy) · Chapter 2 (pandas: load, explore, clean) · pandas analysis (TechStore project level)

Everything you need: syntax, explanation, examples *with the output*, traps, and the questions she likes to ask.

> **How the DS works:** part **on paper** (theory, predict the output, write code by hand, explain a decision) + part **on PC** (a CSV to load, explore, clean, analyse in a notebook). This document covers both. Sections marked ★ are the most likely exam material.

Contents

1. Theory of Chapter 0 (software, IS, data science) ★
2. Variables, types, conversions ★
3. print(), f-strings, input() ★
4. Operators ★
5. if / elif / else ★
6. for, range, while, break/continue ★
7. Lists & slicing ★
8. Tuples, dictionaries, sets ★
9. Nested lists (grids)
10. Strings methods
11. Functions, lambda, comprehensions
12. Libraries & import ★
13. NumPy ★
14. pandas: Series & DataFrame, loading ★
15. pandas: exploring ★
16. pandas: selecting & filtering ★
17. pandas: cleaning (missing, duplicates, types, text, outliers) ★
18. pandas: export & cleaning workflow ★
19. pandas: new columns, apply/map/lambda
20. pandas: groupby, merge, pivot, dates, stats, plots
21. Classic errors & traps ★
22. Recap Q&A (paper part) ★
23. Practical-part checklist ★

1. Theory of Chapter 0 ★

Software

Software = a set of instructions (code) that tells a computer exactly what to do, step by step.

Application software	System software
Programs you use directly: Instagram, Word, a calculator, a game.	Runs in the background so everything else works: Windows, macOS, Android.

Software Engineering — the 6 phases of the lifecycle

Software engineering = the discipline of designing, building, testing and maintaining software in an organised, reliable way (not just "writing some code").

#	Phase	Question
1	Requirements	What should it do?
2	Design	How will it be built?
3	Implementation	Write the code
4	Testing	Does it work correctly?
5	Deployment	Release it to users
6	Maintenance	Fix bugs, add features

Why it matters: banking (a bug sends money to the wrong person), hospitals (records must be accurate, available and secure), flight booking (a crash = lost seat), cars, elevators, traffic lights...

Information System (IS) — 5 components

An IS is **not just software**: it combines **Hardware + Software + Data + People + Process** to produce useful information.

Component	Meaning	School portal example
Hardware	physical machines (computers, servers, phones)	the servers
Software	programs that run on the hardware	the website
Data	raw facts: names, numbers, dates, transactions	the grades
People	everyone who uses or maintains it	students, staff
Process	the steps followed to get work done	how grades are entered

Food delivery app → *Data* = your order, your address, the restaurant's menu prices.

Where data science fits: Collect → Store → Analyse → Decide

1 the IS **collects** data (orders, clicks, sensors) · 2 data is **stored** in files/databases · 3 we use **Python to analyse** it (this course) · 4 people **decide** better.

Why Python? How does it run?

- Reads like English, beginner-friendly, huge community, extremely versatile (data science, web — Instagram/Spotify servers, automation, AI/ML, games).
- Java needs a class + `main` + `System.out.println` (4 lines) — Python: `print("Hello!")` (1 line).
- Python is **interpreted**: you write code → the **interpreter** reads it **line by line, top to bottom** → you see the result immediately. No separate compile step → notebooks (Jupyter/Colab) are interactive.

2. Variables, types, conversions ★

A **variable** is a named box holding a value. No type declaration: Python figures it out (**dynamic typing**). `=` assigns.

```
name = "Amine"      # str
age = 20            # int
price = 19.90       # float
is_student = True   # bool  (capital T / F!)
print(name, age, price, is_student)
print(type(age), type(price))
```

```
OUTPUT
Amine 20 19.9 True
<class 'int'> <class 'float'>
```

Type	Meaning	Examples	Convert with
`int`	whole number	20, -5, 1000	`int("7")` → 7, `int(3.9)` → 3 (truncates)
`float`	decimal	19.90, 3.14, -0.5	`float("2.5")` → 2.5, `float(3)` → 3.0
`str`	text	"Amine", 'Hello!'	`str(20)` → "20"
`bool`	True / False	True, False	`bool(0)` → False, `bool("")` → False, `bool("a")` → True
`NoneType`	"no value"	None	a function without return gives None

```
x = 7
y = "7"
z = 3.5
print(type(x)); print(type(y)); print(type(z))
print(x == int(y))   # int("7") -> 7
print(x == y)        # int vs str -> never equal
```

```
OUTPUT
<class 'int'>
<class 'str'>
<class 'float'>
True
False
```

Traps: `int("3.5")` → **ValueError** (use `int(float("3.5"))`). `int("abc")` → ValueError. Names: case-sensitive (`Age` ≠ `age`), cannot start with a digit, no spaces, not a keyword (`for`, `if`, `class` ...). Use clear names: `age` not `a`.

Multiple assignment: `a, b = 1, 2` · swap: `a, b = b, a` · `x += 1` means `x = x + 1` (also `-=`, `*=`, `/=`).

3. print(), f-strings, input() ★

```
print("Hello, world!")
print("Age:", 20)                 # comma -> a space is added between values
print("A", "B", "C", sep=" - ")   # sep = separator between values
print("Line 1", end=" -> ")       # end = what is printed at the end (default "\n")
print("Line 2")
name = "Lina"
print(f"Welcome, {name}!")        # f-string: variables/expressions inside { }
price = 3.14159
print(f"{price:.2f} TND | {7/2} | {2*5}")   # :.2f = 2 decimals
```

```
OUTPUT
Hello, world!
Age: 20
A - B - C
Line 1 -> Line 2
Welcome, Lina!
3.14 TND | 3.5 | 10
```

Other formats: `f"{1234567:,}"` → `1,234,567` · `f"{0.256:.1%}"` → `25.6%` · concatenation: `"Hello, " + name + "!"` (only str + str!).

input() — the #1 trap

```
age_text = input("How old are you? ")   # user types 20
print(type(age_text))                   # ALWAYS a string!
age = int(age_text)                     # convert before maths
print("Next year you'll be", age + 1)
```

```
OUTPUT
<class 'str'>
Next year you'll be 21
```

```
a = "5"
b = "3"
print(a + b)            # string + string = concatenation
print(int(a) + int(b))  # real addition
print(a * 3)            # string repetition
```

```
OUTPUT
53
8
555
```

`"5" + 3` → **TypeError**: can only concatenate str (not "int") to str. Fix: `int("5") + 3` or `"5" + str(3)`.

4. Operators ★

Operator	Meaning	Example → result
`+ - *`	add, subtract, multiply	5 + 2 → 7
`/`	division — **always a float**	7 / 2 → 3.5 · 6 / 2 → 3.0
`//`	integer (floor) division	7 // 2 → 3 · -7 // 2 → -4
`%`	modulo = remainder	7 % 2 → 1 · 10 % 3 → 1
`**`	power	2 ** 3 → 8

Even number test: `n % 2 == 0`. Last digit: `n % 10`. Divisible by 3: `n % 3 == 0`.
Priority: `**` > `* / // %` > `+ -` → `2 + 3 * 2 ** 2` = 2 + 3*4 = **14**. Parentheses first.

Comparison / logic	Meaning	Example → result
`== !=`	equal / not equal	5 == 5 → True
`> < >= <=`	greater / less (or equal)	7 > 10 → False
`and`	both true	(5 > 1) and (2 > 1) → True
`or`	at least one true	(5 > 1) or (2 > 100) → True
`not`	reverses	not (5 > 1) → False
`in`	membership	"a" in "data" → True · 3 in [1,2] → False

```
age = 17
has_ticket = True
print(age >= 18)
print(age >= 18 or has_ticket)
print(age >= 18 and has_ticket)
print(1 < age < 18)        # chained comparison
```

```
OUTPUT
False
True
False
True
```

`=` assigns, `==` compares. In pandas/NumPy masks we use `& | ~` instead of `and or not` (see §16).

5. if / elif / else ★

Python checks conditions **top to bottom**; the **first True** block runs, the rest are skipped. `else` catches everything else. The **colon** `:` and the **indentation** (4 spaces) are mandatory.

```
temperature = 8
if temperature > 30:
    print("It's hot!")
elif temperature > 15:
    print("Nice weather.")
elif temperature > 0:
    print("A bit cold.")
else:
    print("Freezing!")
```

```
OUTPUT
A bit cold.
```

```
score = 55
if score >= 60:
    print("Pass")
elif score >= 50:
    print("Retake")      # first True condition
else:
    print("Fail")

# one-line (ternary) form
status = "adult" if age >= 18 else "minor"
```

```
OUTPUT
Retake
```

Order matters: test the most specific case first. In "even/odd/zero", check `n == 0` *before* `n % 2 == 0`, otherwise 0 is printed as "Even". Two separate `if`s can both run; `if/elif` runs only one.

6. Loops: for, range, while ★

range	gives
`range(5)`	0, 1, 2, 3, 4 (5 numbers, starts at 0, **stop excluded**)
`range(1, 11)`	1 … 10
`range(0, 10, 2)`	0, 2, 4, 6, 8 (step 2)
`range(5, 0, -1)`	5, 4, 3, 2, 1

```
total = 0
for i in range(1, 5):
    total = total + i
    print("i =", i, "total =", total)
```

```
OUTPUT
i = 1 total = 1
i = 2 total = 3
i = 3 total = 6
i = 4 total = 10
```

```
fruits = ["apple", "banana", "cherry"]
for f in fruits:                        # loop directly on the items
    print("I like", f)
for i, f in enumerate(fruits):          # index + item
    print(i, f)
total = 0
for i in range(1, 101):
    total += i
print(total)                            # 5050
```

```
OUTPUT
I like apple
I like banana
I like cherry
0 apple
1 banana
2 cherry
5050
```

while, break, continue

```
n = 1
while n <= 3:          # repeats while the condition is True
    print(n)
    n += 1             # forget this -> infinite loop!

for i in range(10):
    if i == 2:
        continue       # skip the rest of THIS iteration
    if i == 4:
        break          # leave the loop completely
    print(i)
```

```
OUTPUT
1
2
3
0
1
3
```

Patterns to know by heart: **running total** (`total += x`), **counter** (`count += 1` inside an if), **max by hand** (`best = lst[0]`, then `if x > best: best = x`), **multiplication table** (`for i in range(1, 11): print(n, "x", i, "=", n*i)`).

7. Lists ★

Ordered, **mutable** (can change), can hold mixed types. Indexing starts at **0**; negative index counts from the end.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0], fruits[-1], len(fruits))
fruits.append("mango")        # add at the end
fruits[0] = "pear"            # change an item
print(fruits)
print("kiwi" in fruits)
```

```
OUTPUT
apple cherry 3
['pear', 'banana', 'cherry', 'mango']
False
```

Operation	Code	Result / note
add at end	`lst.append(x)`	modifies the list, returns None
insert at position	`lst.insert(1, x)`	
add several	`lst.extend([a, b])` / `lst + [a, b]`	
remove by value	`lst.remove(x)`	first occurrence; ValueError if absent
remove by index	`lst.pop()` / `lst.pop(0)` / `del lst[0]`	pop returns the item
sort	`lst.sort()` (in place) / `sorted(lst)` (new list)	`reverse=True` for descending
stats	`len, sum, min, max`	`sum(lst)/len(lst)` = average
count / find	`lst.count(x)` / `lst.index(x)`	

Slicing `lst[start:stop:step]` — stop is excluded

```
n = [10, 20, 30, 40, 50]
print(n[1:3])    # index 1, 2
print(n[:2])     # first 2
print(n[2:])     # from index 2 to the end
print(n[-2:])    # last 2
print(n[::2])    # every 2nd
print(n[::-1])   # reversed
```

```
OUTPUT
[20, 30]
[10, 20]
[30, 40, 50]
[40, 50]
[10, 30, 50]
[50, 40, 30, 20, 10]
```

`b = a` does **not** copy a list: both names point to the same list (changing b changes a). Copy with `b = a.copy()` or `a[:]`. `lst[10]` on a short list → **IndexError**.

8. Tuples, dictionaries, sets ★

Tuple — like a list but locked (immutable)

```
point = (3, 7)
x, y = point          # unpacking
print(x, y)
# point[0] = 5  -> TypeError!
single = (5,)         # 1-element tuple
needs a comma
```

```
OUTPUT
3 7
```

Use for **fixed data**: coordinates, RGB colours, `df.shape` (a tuple!).

List vs tuple

List `[ ]`	Tuple `( )`
mutable	immutable
append, remove...	only count, index
data that changes	fixed data, safer, a bit faster

Dictionary — key → value pairs

```
student = {"name": "Salma", "age": 21, "grade": "A"}
print(student["name"])          # access by KEY, not by number
student["city"] = "Tunis"       # add (or update) a key
student["age"] = 22             # update
for key, value in student.items():
    print(key, "->", value)
print(student.get("phone", "N/A"))   # safe access with default
print(list(student.keys()), len(student))
```

```
OUTPUT
Salma
name -> Salma
age -> 22
grade -> A
city -> Tunis
N/A
['name', 'age', 'grade', 'city'] 4
```

`.keys()`, `.values()`, `.items()`, `"age" in student` (tests keys), `del student["city"]`, `student.pop("age")`. Keys are unique; `student["phone"]` on a missing key → **KeyError**.

```
# Counting with a dict (classic exam exercise)
words = ["tunis", "sfax", "tunis", "sousse", "tunis"]
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)
```

```
OUTPUT
{'tunis': 3, 'sfax': 1, 'sousse': 1}
```

Set `{1, 2, 3}`: no duplicates, no order → `set([1,1,2])` → `{1, 2}`; `len(set(lst))` = number of distinct values.

9. Nested lists — a grid (Tic-Tac-Toe)

```
board = [["_", "_", "_"],
         ["_", "_", "_"],
         ["_", "_", "_"]]
board[1][1] = "X"          # board[row][col] -> centre
board[0][2] = "O"          # top-right
for row in board:          # each row is a list
    print(" | ".join(row))
```

```
OUTPUT
_ | _ | O
_ | X | _
_ | _ | _
```

`board[1]` = the whole middle row (a list); `board[1][2]` = one cell. Whose turn: `player = "X" if turn % 2 == 0 else "O"`. Empty cell check: `if board[r][c] == "_":`.

10. Strings

```
s = "  Hello Data  "
print(s.strip())            # remove spaces at both ends
print(s.lower(), s.upper())
print("tunis city".title()) # first letter of each word in capital
print("a,b,c".split(","))   # str -> list
print("-".join(["a", "b"])) # list -> str
print("Hello".replace("l", "L"), len("Hello"), "Hello"[1:3])
print("data" in "big data", "abc".startswith("a"), "42".isdigit())
```

```
OUTPUT
Hello Data
  hello data     HELLO DATA
Tunis City
['a', 'b', 'c']
a-b
HeLLo 5 el
True True True
```

Strings are **immutable**: `s.upper()` returns a new string, you must re-assign `s = s.upper()`. Same idea in pandas: `df["c"] = df["c"].str.strip()`.

11. Functions, lambda, comprehensions

```
def total_price(price, quantity, discount=0):     # discount has a default value
    """Return the price after discount."""
    total = price * quantity
    return total * (1 - discount / 100)

print(total_price(10, 3))           # 30.0
print(total_price(10, 3, 10))       # 27.0
print(total_price(quantity=2, price=5))   # keyword arguments -> 10.0
```

```
OUTPUT
30.0
27.0
10.0
```

`return` sends a value back and **stops** the function; `print` only displays. A function without return returns `None`. Variables created inside a function are **local**.

```
square = lambda x: x ** 2           # anonymous one-line function
print(square(4))                    # 16
squares = [x ** 2 for x in range(5)]            # list comprehension
evens = [x for x in range(10) if x % 2 == 0]
prices = {"pen": 1.5, "book": 12}
expensive = {k: v for k, v in prices.items() if v > 5}
print(squares, evens, expensive)
```

```
OUTPUT
16
[0, 1, 4, 9, 16] [0, 2, 4, 6, 8] {'book': 12}
```

Error handling (bonus): `try: n = int(input()) except ValueError: print("Not a number")`.

12. Libraries & import ★

A **library** = a toolbox of ready-made, tested code → saves time, reliable.

```
import random
print(random.randint(1, 100))     # random int, 1 and 100 INCLUDED
print(random.choice(["a", "b"]))  # random element
lst = [1, 2, 3]; random.shuffle(lst)

import math
print(math.sqrt(16))              # 4.0 (always a float)
print(math.floor(3.7), math.ceil(3.2), round(3.14159, 2))

from math import pi               # import just one piece -> use pi directly
print(pi)

import numpy as np                # alias
import pandas as pd
```

```
OUTPUT
(random value)
(random value)
4.0
3 4 3.14
3.141592653589793
```

Other libraries met: `datetime` (dates), `matplotlib` (charts), `numpy`, `pandas`.

13. NumPy (Chapter 1 + Lab 1) ★

NumPy = the base library for numerical computing; pandas is built on it. Its object is the **ndarray**.

Python list	NumPy array
can contain **different** types: `["beep", False, 56, .945, [3, 2, 5]]`	only **one** data type (homogeneous)
slower, more memory	less memory, much faster
`[1,2] * 2` → `[1, 2, 1, 2]`	`np.array([1,2]) * 2` → `[2 4]` (**vectorised**, element by element)

Creating arrays

```
import numpy as np
arr = np.array([3, 2, 5, 8, 4, 9, 7, 6, 1])        # 1D from a list
print(type(arr))
m = np.array([[3, 2, 5], [9, 7, 1], [4, 3, 6]])     # 2D from a list of lists
print(np.zeros((2, 3)))                              # shape passed as a TUPLE
print(np.arange(-3, 4))       # like range: stop excluded
print(np.arange(4), np.arange(-3, 4, 3))
r = np.random.random((2, 4))   # floats in [0, 1), shape 2x4
```

```
OUTPUT
<class 'numpy.ndarray'>
[[0. 0. 0.]
 [0. 0. 0.]]
[-3 -2 -1  0  1  2  3]
[0 1 2 3] [-3  0  3]
```

Also: `np.ones((2,2))`, `np.linspace(0, 1, 5)` → `[0. 0.25 0.5 0.75 1.]`, `np.full((2,2), 7)`.

Dimensions & shape

1D = vector · 2D = matrix · 3D or more = tensor. In 2D: rows are the 1st dimension, columns the 2nd. A 3D array = an array of 2D arrays (`np.array([a2d, b2d, c2d])` → shape `(3, 2, 2)`).

```
array = np.zeros((3, 5))
print(array.shape, array.ndim, array.size)     # (rows, cols), nb of dims, nb of elements
a = np.array([[1, 2], [5, 7], [6, 6]])
print(a.flatten())                 # -> 1D
print(a.reshape((2, 3)))           # same 6 elements, new shape
# a.reshape((3, 3)) -> ValueError: cannot reshape array of size 6 into shape (3,3)
```

```
OUTPUT
(3, 5) 2 15
[1 2 5 7 6 6]
[[1 2 5]
 [7 6 6]]
```

reshape rule: the product of the new shape must equal `size` (6 = 2×3 = 3×2 = 6×1). `reshape(-1)` = flatten; `reshape(2, -1)` lets NumPy compute the missing dimension.

Data types (dtype)

Python	NumPy	note
int	`np.int64`, `np.int32`	int64 = default for integers
float	`np.float64`, `np.float32`	float64 = default for decimals
str	`<U12`	Unicode string, max 12 characters
bool	`np.bool_`	

Bits: `int32` uses 32 bits → can store 2^{32} = **4,294,967,296** different integers; `int64` uses 64 bits (bigger numbers, more memory). Smaller dtype = less memory.

```
print(np.array([1.32, 5.78, 175.55]).dtype)
print(np.array([[1, 2, 3], [4, 5, 6]]).dtype)
print(np.array(["Introduction", "to", "NumPy"]).dtype)       # longest word = 12 chars
f32 = np.array([1.32, 5.78, 175.55], dtype=np.float32)      # choose the dtype
print(f32.dtype)
b = np.array([[True, False], [False, False]], dtype=np.bool_)
print(b.astype(np.int32))                                    # conversion
```

```
OUTPUT
float64
int64
<U12
float32
[[1 0]
 [0 0]]
```

Type coercion ★ — an array has ONE type, so NumPy converts everything to the most general one

```
print(np.array([True, "Boop", 42, 42.42]))   # a string present -> everything becomes str
print(np.array([0, 42, 42.42]).dtype)        # int + float -> float
print(np.array([True, False, 42]).dtype)     # bool + int -> int
```

```
OUTPUT
['True' 'Boop' '42' '42.42']
float64
int64
```

Hierarchy: **bool → int → float → str** (the "biggest" type present wins).

Indexing, slicing, statistics, masks (Lab 1)

```
days = np.array(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
sales = np.array([120, 150, 95, 180, 160, 210, 175])
print(sales[0], sales[-1], sales[1:5])          # first, last, Tue..Fri
print(sales.sum(), sales.mean().round(2), sales.min(), sales.max())
print(sales.argmax(), days[sales.argmax()])     # INDEX of the max -> use it on days
mask = sales > 150                              # boolean array
print(mask)
print(sales[mask], days[mask], mask.sum())      # filter + count (True = 1)
print(days[sales >= sales.mean()])
print(days[(sales > 100) & (sales < 170)])      # & | ~ with parentheses
```

```
OUTPUT
120 175 [150  95 180 160]
1090 155.71 95 210
5 Sat
[False False False  True  True  True  True]
[180 160 210 175] ['Thu' 'Fri' 'Sat' 'Sun'] 4
['Thu' 'Fri' 'Sat' 'Sun']
['Mon' 'Tue' 'Fri']
```

```
m = np.array([[1, 2, 3], [4, 5, 6]])
print(m[0, 1], m[1], m[:, 0])     # row 0 col 1 | row 1 | column 0
print(m.sum(axis=0), m.sum(axis=1))   # axis=0 -> per column, axis=1 -> per row
print(m * 10, m + m)              # vectorised maths
print(np.sort(np.array([3, 1, 2])), np.median(sales), sales.std().round(2))
```

```
OUTPUT
2 [4 5 6] [1 4]
[5 7 9] [ 6 15]
[[10 20 30]
 [40 50 60]] [[ 2  4  6]
 [ 8 10 12]]
[1 2 3] 160.0 35.7
```

Also: `np.where(sales > 150, "good", "low")`, `np.unique()`, `np.round()`, `argmin()`, `cumsum()`. Plot: `plt.scatter(np.arange(0, 7), np.arange(-3, 4)); plt.show()`.

14. pandas — Series, DataFrame, loading (Chapter 2) ★

DataFrame = a table; each **column is a Series** with **one dtype**. The **index** = row labels (0, 1, 2…). Think: a spreadsheet, or a dictionary of lists.

```
import pandas as pd
df = pd.DataFrame({"city": ["Tunis", "Sfax", "Sousse"], "quantity": [1, 3, 2]})
print(df)
print(type(df["city"]))         # one column -> Series
print(df.loc[0, "city"])        # one cell by label
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s["b"], s.mean())
```

```
OUTPUT
     city  quantity
0   Tunis         1
1    Sfax         3
2  Sousse         2
<class 'pandas.core.series.Series'>
Tunis
20 20.0
```

Loading

```
orders = pd.read_csv("data/orders.csv")                 # CSV -> DataFrame
# pd.read_csv("f.csv", sep=";")                         # other separator
# pd.read_csv("f.csv", parse_dates=["order_date"])      # read dates directly
# pd.read_excel("file.xlsx", sheet_name="Sales")        # Excel (needs openpyxl)
# also: pd.read_json, pd.read_sql, pd.read_parquet
# Colab: from google.colab import files; files.upload()  -> file goes to /content
```

`read_csv` splits each line into columns, **infers the dtype**, and turns empty cells and texts like `N/A`, `NaN`, `null` into missing values (`NaN`).

15. pandas — exploring ("always look before you clean") ★

Code	What it shows
`df.head(10)` / `df.tail(5)`	first / last rows (default 5)
`df.shape`	tuple `(rows, columns)` e.g. `(12060, 10)` — attribute, **no ()**
`df.size`	number of cells = rows × columns
`len(df)`	number of rows
`df.columns` / `df.dtypes`	column names / type of each column
`df.info()`	**types + non-null counts** (→ missing values) + memory
`df.describe()`	count, mean, std, min, 25%, 50%, 75%, max of numeric columns → spot impossible values (quantity max = 999)
`df.describe(include="all")`	also text columns (unique, top, freq)
`df["status"].value_counts()`	frequency of each category (`normalize=True` → proportions, `dropna=False` → count NaN too)
`df["city"].unique()` / `.nunique()`	distinct values / how many (`df.nunique()` for all columns)
`df["q"].mean() .median() .min() .max() .sum() .std()`	column statistics (NaN ignored)
`df["price"].idxmax()`	index label of the max → `df.loc[df["price"].idxmax()]`

> dtypes in pandas: `int64`, `float64` (also used when an int column has NaN), `object` / `str` (text — pandas 3 uses a dedicated `str` dtype), `bool`, `datetime64[ns]`, `category`.

16. pandas — selecting & filtering ★

```
orders["city"]                                  # one column -> Series
orders[["order_id", "city", "status"]]          # several columns -> DataFrame (double
brackets!)
orders.drop(columns="status")                   # all except one
orders.loc[0, "city"]                           # cell by LABEL
orders.loc[0:4, ["order_id", "city"]]           # labels 0..4 INCLUDED
orders.iloc[0:5, 0:3]                           # POSITIONS 0..4 (5 excluded), first 3
columns
orders.iloc[-1]                                 # last row
```

	`[]`	`.loc[rows, cols]`	`.iloc[rows, cols]`
selects by	column name(s) or a boolean mask (rows)	**labels** (index, column names) or mask	**integer positions**
slice end	—	**included**	**excluded**

Boolean masks

```
mask = (
    (orders["status"] == "Delivered")
    & (orders["city"] == "Tunis")
    & (orders["quantity"] >= 3)
)
orders.loc[mask, ["order_id", "quantity"]]           # filter rows + choose columns
orders[mask]                                         # same rows, all columns
orders.query("status == 'Delivered' and quantity >= 3")   # readable alternative

orders[orders["city"].isin(["Tunis", "Sfax"])]       # several values (= OR)
orders[orders["quantity"].between(2, 5)]             # 2 <= q <= 5 (both included)
orders[~(orders["status"] == "Returned")]            # ~ = NOT
orders[(orders["city"] == "Tunis") | (orders["city"] == "Sfax")]
orders[orders["payment_method"].isna()]              # rows where a value is missing
len(orders[mask]) ;  mask.sum()                      # count the rows
```

Rules: combine with `&` (and), `|` (or), `~` (not) — **never** `and/or` on columns (→ ValueError: truth value of a Series is ambiguous). Put **parentheses around every condition** (`&` is evaluated before `==`).

Sorting

```
orders.sort_values("unit_price", ascending=False).head(5)       # top 5 prices
orders.sort_values(["city", "quantity"], ascending=[True, False])
orders.nlargest(5, "unit_price") ; orders.nsmallest(3, "quantity")
orders.sort_index() ; orders.reset_index(drop=True)             # rebuild 0..n-1 index
```

17. pandas — cleaning ★★

Missing values: choose, don't guess

```
orders.isna().sum()                                    # missing per column
(orders.isna().mean() * 100).sort_values(ascending=False)   # % missing per column
orders.isna().sum().sum()                              # total missing
orders[orders.isna().any(axis=1)]                      # rows with at least one NaN
orders["payment_method"] = orders["payment_method"].fillna("Unknown")    # label
orders["age"] = orders["age"].fillna(orders["age"].median())             # median
orders["region"] = orders["region"].fillna(orders["region"].mode()[0])   # most frequent
orders = orders.dropna(subset=["order_date"])          # drop rows missing a KEY field
orders = orders.drop(columns=["useless_col"])          # drop a column
```

Situation	Typical action	Example
missing category	fill with a label (or the mode)	`payment_method` → "Unknown"
missing number, a business rule exists	impute (median of the group)	`unit_price` → median price of the same product
missing key field	drop the rows	`order_date` missing → cannot place it in time
many missing in a column	ask the business; maybe drop the column	age missing for 3 % of customers

Never replace all missing values by 0: it creates fake data and biases the averages (a price of 0, an age of 0 don't exist). **Median** > mean when there are outliers (robust). **Document every choice.** `isna()` = `isnull()` (aliases); `notna()` = the opposite.

Duplicates

```
orders.duplicated().sum()                  # fully identical rows (60 in Lab 2)
orders[orders.duplicated(keep=False)]      # show all copies
orders = orders.drop_duplicates()          # keep the first occurrence
orders["order_id"].duplicated().sum()      # duplicated IDs (different problem!)
orders["order_id"].is_unique
orders.drop_duplicates(subset=["order_id"], keep="last")
```

Duplicated row = identical in every column → safe to delete. **Duplicated ID** = same key but maybe different content → investigate. Not removing duplicates → revenue counted twice.

Data types

```
orders["order_date"] = pd.to_datetime(orders["order_date"], errors="coerce")   # invalid -> NaT
orders = orders.dropna(subset=["order_date"])
orders["quantity"] = orders["quantity"].astype("int64")   # only AFTER NaN are handled
orders["price"] = pd.to_numeric(orders["price"], errors="coerce")   # text -> number, bad -> NaN
orders["city"] = orders["city"].astype("category")
orders.dtypes
```

`errors="coerce"` turns invalid values into missing (`NaT` for dates, `NaN` for numbers) instead of crashing. The right types unlock the `.dt` and `.str` tools. A column with NaN cannot be `int64` (NaN is a float) → that's why quantity is read as `1.0, 2.0`.

Text cleaning

```
orders["city"].nunique()                                    # 50 spellings!
orders["city"] = orders["city"].str.strip().str.title()     # ' tunis ' / 'TUNIS' ->
'Tunis'
orders["city"].nunique()                                    # 10
orders["city"] = orders["city"].replace({"Gabes": "Gabès"}) # fix a specific typo
orders["email"].str.lower() ; orders["name"].str.contains("Ben") ; orders["code"].str[:3]
```

Outliers — business rule beats statistics

```
orders["quantity"].value_counts().sort_index()        # see the 999
orders = orders[orders["quantity"] <= 10].copy()      # rule: max 10 units per line
# statistical rule (IQR):
q1, q3 = orders["unit_price"].quantile([0.25, 0.75])
iqr = q3 - q1
outliers = orders[(orders["unit_price"] < q1 - 1.5 * iqr) | (orders["unit_price"] > q3 +
1.5 * iqr)]
```

Ask: is 999 a real order or a data-entry error? Who can confirm?

Group-wise imputation with transform ★

```
by_product = orders.groupby("product_id")["unit_price"]
median = by_product.transform("median")        # a column ALIGNED with the rows (same
length)
orders["unit_price"] = orders["unit_price"].fillna(median)
```

`groupby(...).median()` → one value per product (short table).
`groupby(...).transform("median")` → the product's median repeated on **every row** → perfect for `fillna`.

18. Export, validation & the cleaning workflow ★

```
orders.to_csv("orders_clean.csv", index=False)          # index=False: no 'Unnamed: 0'
column
orders.head(500).to_excel("orders_sample.xlsx", index=False)
check = pd.read_csv("orders_clean.csv", parse_dates=["order_date"])
assert check.shape == orders.shape                      # validate: stops with an error if
False
assert orders["unit_price"].isna().sum() == 0
print(f"Removed: {(12060 - len(orders)) / 12060 * 100:.1f} %")
```

1 Raw file	2 Explore	3 Decide	4 Clean	5 Validate	6 Export
keep it untouched	head, info, describe, value_counts	drop, fill or fix? write it down	code the decisions in order	assert / counts before–after	new file: clean CSV / Excel

Lab 2 result: 12,060 raw rows → 60 duplicates → 12,000 → 20 bad dates → 11,980 → 15 quantities of 999 → **11,965 clean rows (0.8 % removed)**.

A reusable cleaning function

```
def clean_orders(raw):
    df = raw.drop_duplicates().copy()
    df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")
    df = df.dropna(subset=["order_date"])
    df = df[df["quantity"] <= 10]
    df["payment_method"] = df["payment_method"].fillna("Unknown")
    return df

orders = clean_orders(pd.read_csv("data/orders.csv"))   # same input -> same output, every
night
```

Modern pandas good habits (pandas 3)

- **Copy-on-Write**: a filtered DataFrame is a safe copy; add `.copy()` when you plan to edit it (avoids *SettingWithCopyWarning* in pandas 2).
- **String dtype**: text columns use a dedicated `str` dtype by default.
- **Avoid `inplace=True`**: assign the result back: `df["a"] = df["a"].fillna(0)`.
- **Method chaining**: `df.assign(total=df.q * df.p).query("total > 100").sort_values("total")`.
- **Keep the raw data**: work on `clean = raw.copy()`.

Industry: banking (duplicated transactions → wrong balances, false fraud alerts), healthcare (different date formats → wrong patient timelines), telecom (missing usage records → under-billing), retail (typos in city names → wrong regional reports).

19. pandas — new columns, conditions, apply / map / lambda

```
df["Gross_Revenue"] = df["Quantity"] * df["Unit_Price"]          # vectorised, row by row
df["Net_Revenue"] = df["Gross_Revenue"] * (1 - df["Discount"] / 100)
df["Profit"] = df["Net_Revenue"] - df["Quantity"] * df["Cost_Price"]
df["Margin"] = np.where(df["Net_Revenue"] != 0, df["Profit"] / df["Net_Revenue"] * 100,
np.nan)  # avoid /0
df = df.assign(Tax=df["Net_Revenue"] * 0.05)                     # same, chainable
df = df.rename(columns={"qty": "Quantity"})
```

Conditional classification — 3 ways

```
import numpy as np
df["Sign"] = np.where(df["Profit"] >= 0, "Positive", "Negative")        # 2 classes

conds = [df["Profit"] > 500, df["Profit"] >= 100, df["Profit"] >= 0]   # first True wins
(like elif)
df["Level"] = np.select(conds, ["High", "Medium", "Low"], default="Loss")

df["Age_Group"] = pd.cut(df["Age"], bins=[0, 29, 49, 120], labels=["Young", "Adult",
"Senior"])  # intervals (a, b]
```

apply, map, lambda

```
df["Tax"] = df["Net_Revenue"].apply(lambda x: x * 0.05)          # function on each VALUE
def age_group(age):
    if age < 30: return "Young"
    elif age < 50: return "Adult"
    return "Senior"
df["Age_Group"] = df["Age"].apply(age_group)
df["Pay"] = df["Payment"].map({"Cash": "Cash at counter", "Online": "Online payment"})  #
dict (missing key -> NaN)
df["Margin2"] = df.apply(lambda row: row["Profit"] / row["Net_Revenue"] * 100, axis=1)  #
each ROW
```

map (Series) = replace each value via a dict/Series/function. **apply** = run a function on each value (Series) or each row/column (DataFrame, `axis=1` = rows). **lambda** = small anonymous function. Use `def` when the logic has several steps or must be reused. Vectorised code (`*`, `np.where`) is faster than `apply`.

20. pandas — groupby, merge, pivot, dates, statistics, plots

groupby → split / apply / combine

```
df.groupby("Region")["Net_Revenue"].sum()                          # one stat
df.groupby("Region")["Net_Revenue"].agg(["sum", "mean", "count"])   # several
df.groupby("Region").agg(                                          # named aggregation ⋆
    Revenue=("Net_Revenue", "sum"),
    Profit=("Profit", "sum"),
    Orders=("Order_ID", "count"),
    Avg_Discount=("Discount", "mean"),
).sort_values("Revenue", ascending=False)
df.groupby(["Region", "Category"])["Profit"].sum().reset_index()  # 2 keys -> back to a
table
share = df.groupby("Region")["Net_Revenue"].sum() / df["Net_Revenue"].sum() * 100   # %
contribution
df.groupby("Region")["Profit"].sum().rank(ascending=False)       # ranking
df.groupby("Region")["Profit"].sum().idxmax()                    # best group
```

Category columns: add `observed=True` to `groupby` to avoid empty groups.

merge (join two tables on a key)

```
full = sales.merge(products, on="Product_ID", how="left")     # keep ALL sales rows
# how: "inner" (only matching keys), "left", "right", "outer" (all)
# different key names: left_on="prod", right_on="Product_ID"
sales["Product_ID"].isin(products["Product_ID"]).all()       # every sale has a product?
price = sales["Product_ID"].map(products.set_index("Product_ID")["List_Price"])  # lookup
without merge
pd.concat([df_jan, df_feb])                                  # stack tables vertically
```

Dates (.dt)

```
d = df["Order_Date"].dt          # column must be datetime64 first!
df["Year"], df["Month"], df["Day"] = d.year, d.month, d.day
df["Month_Name"], df["Day_Name"], df["Quarter"] = d.month_name(), d.day_name(), d.quarter
df["Is_Weekend"] = d.dayofweek >= 5          # Monday = 0 ... Sunday = 6
df[(df["Order_Date"] >= "2025-01-01") & (df["Order_Date"] <= "2025-03-31")]   # date filter
df["Season"] = df["Month"].map({12: "Winter", 1: "Winter", 2: "Winter", 3: "Spring"})  #
...
(df["Order_Date"].max() - df["Order_Date"].min()).days
df.set_index("Order_Date").resample("ME")["Net_Revenue"].sum()   # monthly total ("M" in
pandas < 2.2)
```

pivot_table & crosstab

```
pd.pivot_table(df, values="Net_Revenue", index="Category", columns="Quarter",
aggfunc="sum", margins=True)
pd.crosstab(df["Region"], df["Payment_Method"])                    # counts
pd.crosstab(df["Region"], df["Payment_Method"], normalize="index") # % per row
```

Statistics

```
df["Profit"].describe()
df["Profit"].agg(["mean", "median", "min", "max", "std", "var"])
df["Profit"].quantile([0.25, 0.5, 0.75])      # Q1, median, Q3
df[["Quantity", "Unit_Price", "Profit"]].corr()   # correlation matrix (-1 .. +1)
```

Mean sensitive to extremes; **median** = middle value, robust. Mean > median → right-skewed (few big values). **Std** = typical distance to the mean. **Q3 (75th percentile)** = 75 % of the values are below. **Correlation ≠ causation.**

Plots (matplotlib / pandas)

```
import matplotlib.pyplot as plt
df.groupby("Region")["Net_Revenue"].sum().plot(kind="bar", title="Revenue by region")
plt.show()
monthly.plot(kind="line", marker="o") ; df["Age"].plot(kind="hist", bins=10)
plt.scatter(df["Unit_Price"], df["Profit"]); plt.xlabel("Price"); plt.ylabel("Profit");
plt.show()
df["Category"].value_counts().plot(kind="pie", autopct="%1.0f%%")
```

21. Classic errors & traps ★

Error	Typical cause	Fix
`SyntaxError`	missing `:` after if/for/def, unclosed bracket/quote, `=` instead of `==` in an if	check the line (and the one before)
`IndentationError`	block not indented / mixed spaces	4 spaces under if/for/def
`NameError`	variable not defined / typo / cell not run / string without quotes	define it, run cells in order
`TypeError`	`"5" + 3`, calling `len(5)`, modifying a tuple, mean of a text column	convert types
`ValueError`	`int("abc")`, `int("3.5")`, bad reshape, `and` between Series	check the value / use `&` |
`IndexError`	`lst[5]` on a 3-item list	valid indices: 0 ... len-1
`KeyError`	dict key or DataFrame column name doesn't exist (case! spaces!)	check `df.columns`
`ZeroDivisionError`	`x / 0`	test before dividing / `np.where`
`FileNotFoundError`	wrong path (Colab: `/content/file.csv`)	check the path / upload
`IntCastingNaNError`	`astype(int)` on a column with NaN	fillna / dropna first

- `df.shape`, `df.columns`, `df.dtypes`, `df.size` are **attributes** (no parentheses); `df.info()`, `df.head()`, `df.describe()` are **methods**.
- `df["a", "b"]` is wrong → `df[["a", "b"]]`.
- A pandas method returns a **new** object: `df.dropna()` alone changes nothing → `df = df.dropna()`.
- `loc` slice includes the end; `iloc` and Python slices exclude it. After filtering, labels have gaps → `reset_index(drop=True)`.
- `value_counts()` ignores NaN unless `dropna=False`. `mean()` skips NaN silently.
- `range(1, 10)` stops at 9. `random.randint(1, 10)` includes 10. `np.arange(1, 10)` stops at 9.
- `7 / 2 = 3.5`, `7 // 2 = 3`, `6 / 2 = 3.0` (float!).

22. Recap Q&A — the paper part ★

Question	Answer
6 phases of the software lifecycle?	Requirements, Design, Implementation, Testing, Deployment, Maintenance
5 components of an IS?	Hardware, Software, Data, People, Process
Is Instagram software? Why?	Yes — a program (set of instructions) running on your phone (application software)
4 steps where data science fits?	Collect → Store → Analyse → Decide
How does Python run?	An interpreter reads the code line by line, top to bottom, no separate compile step
4 basic types?	int, float, str, bool — check with `type()`
What type does input() return?	Always `str`
`"5" + "3"`?	`"53"` (concatenation)
`len([1,2,3,4])`?	4
Which type cannot be changed after creation?	tuple (immutable)
Which type for a fixed (x, y) coordinate?	a tuple
Set the top-right cell of a 2×2 board to "O"?	`board[0][1] = "O"`
Why use a library?	tested, reliable code that already exists → saves time
Python list vs NumPy array?	list: mixed types, slower; array: one type, less memory, fast, vectorised
Matrix vs tensor?	matrix = 2 dimensions; tensor = 3 or more
`np.array([0, 42, 42.42]).dtype`?	`float64` (type coercion)
What does `.flatten()` do? `.reshape((3,3))` on 6 elements?	returns a 1D copy; ValueError (size must stay the same)
How many integers can int32 store?	2^{32} = 4,294,967,296
Series vs DataFrame?	Series = one column (1D, labelled); DataFrame = table of Series sharing the same index
Which method shows types and missing counts?	`df.info()`
Why not fill all missing values with 0?	it creates fake values and biases statistics
What does `errors="coerce"` do in `to_datetime`?	turns invalid dates into missing values (NaT) instead of an error
Fill a missing price with the product's median?	`df["p"].fillna(df.groupby("product_id")["p"].transform("median"))`
`loc` vs `iloc`?	labels (end included) vs integer positions (end excluded)
Why parentheses in masks?	`&` / `|` have higher priority than `==`, `>`…
shape vs size?	(rows, cols) vs rows × cols

Why `index=False`?	to not write the index as an extra column (`Unnamed: 0`)
Why keep the raw file untouched?	reproducibility, possibility to redo/check the cleaning, audit
Duplicated row vs duplicated ID?	identical everywhere (delete) vs same key, different content (investigate)
Why did `quantity` load as float (1.0)?	it contains NaN, and NaN is a float
Business rule vs statistical rule for outliers?	the business rule (e.g. max 10 units) is more reliable: it knows what is *possible*
transform vs agg?	transform returns a column aligned with the rows; agg returns one value per group

23. Practical part — checklist (do it in this order) ★

```
# 0. imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. load + look
df_raw = pd.read_csv("file.csv")          # Colab: files.upload() then "/content/file.csv"
df = df_raw.copy()                        # keep the raw data!
df.head(); df.shape; df.info(); df.describe(); df.nunique()
for col in df.select_dtypes(exclude="number").columns:   # text columns
    print(col, df[col].unique()[:20])

# 2. missing values
(df.isna().mean() * 100).sort_values(ascending=False)
# decide column by column -> fillna(label / median / mode / group median) or dropna(subset=
[...])

# 3. duplicates
df.duplicated().sum(); df = df.drop_duplicates()

# 4. types & text
df["date"] = pd.to_datetime(df["date"], errors="coerce")
df["col"] = df["col"].str.strip().str.title()
df["qty"] = df["qty"].astype("int64")

# 5. impossible values (business rules)
df = df[(df["qty"] > 0) & (df["qty"] <= 10)].copy()

# 6. validate
assert df[["date", "qty"]].isna().sum().sum() == 0

# 7. analyse
df["revenue"] = df["qty"] * df["price"]
df.groupby("city")["revenue"].agg(["sum", "mean", "count"]).sort_values("sum",
ascending=False)

# 8. export + report
df.to_csv("clean.csv", index=False)
print(len(df_raw), "->", len(df), f"({(len(df_raw) - len(df)) / len(df_raw):.1%} removed)")
```

Marks are won by: printing the result of each step (before → after counts), a **one-line comment per decision** ("# median because outliers", "# drop: key field"), clean variable names, Markdown titles per exercise, and answering the written questions in sentences. Run **Restart & Run all** before submitting.

Built from the Chapter 0, 1, 2 slides, Labs 0–2 and the TechStore project · good luck tomorrow 💪